

Architecture worth keeping

A whole-application review, from system boundaries to implementation seams

tiptotyp is a native desktop application with external language/rendering tools, not a browser application with a thin desktop wrapper. Its architecture combines a small deterministic core, a large native application adapter, and two rendering pipelines. Its strongest decisions concern document correctness, stale-result rejection, native safety boundaries and testability. Its weakest area is **ownership of coordination**: too many independent concerns still meet inside `EditorApp`.

Overall judgment: preserve the platform, core state machines and service choices. Improve the seams between them. This is a modularizing-rewrite problem, not a convincing case for a wholesale rewrite, a new UI framework, or one process per document.

The most valuable architectural improvements

1. Give documents stable storage and identity independent of which tab is active.
2. Consolidate saving into a transaction-oriented, asynchronous coordinator.
3. Separate preview policy, service synchronization and page-resource ownership.
4. Bound background execution, and share immutable workspace results where safe.
5. Extend the existing “read-only inputs plus actions” pattern beyond Settings.

The proposed issues near the end specify scope, exclusions, completion criteria, risks and performance expectations. They are candidate work items, not implemented changes. The companion resource report supplies measurements; this report explains the architecture that produces those costs and constrains possible improvements.

Reading order

Sections 1-3 describe the application from outside: dependencies, startup and windows. Sections 4-11 drill into documents, editing, services, previews, projects, Git and presentation. Section 12 covers cross-cutting guarantees and delivery. Sections 13-16 propose scoped rewrites; section 17 explains how to carry them out. Section 18 provides a module map for navigating the complete application.

Evidence standard. “Current” means inspected source in this checkout. “Risk” means a structural weakness, not necessarily a reproduced bug. “Proposal” means an alternative design whose benefits still need verification. Historical design notes are useful context, but do not override the implemented code.

Audit boundary. First-party architecture, dependency boundaries, representative critical paths, tests and delivery machinery were reviewed. This is not a proof of every code path, a dependency vulnerability audit, or a fresh cross-platform GUI test. Application behavior and `todo.typ` were not changed for this report.

1. The outermost system

The app owns editing, file workflows, windows and presentation. Typst and Tinymist own language compilation/intelligence; the OS owns native windowing, dialogs and webview infrastructure. Local files and repositories remain the user's source of truth. There is no first-party cloud backend or plugin runtime in the reviewed tree.

Boundary	Current responsibility	Assessment
eframe / egui / winit	Native event loop, OpenGL-backed egui surfaces, input, widgets, text layout and viewport management.	Good leverage; some deep text-layout/native integration creates upgrade risk.
Tinymist	LSP over stdio; in-memory document synchronization; interactive preview served locally.	Avoids implementing a compiler or private preview protocol. Session ownership is substantial app logic.
Typst CLI	Canonical PDF artifacts when required, including export and raster fallback.	Clear artifact authority; not always running in parallel with interactive preview.
Poppler	PDF page rasterization and optional link extraction.	Pragmatic fallback; external executable availability and all-page memory need attention.
Wry / OS webview	Embedded interactive preview on macOS and Windows.	Uses Tinymist's frontend. Linux currently takes the raster route.
Filesystem / Git / registry	Documents, project data, Git CLI operations, installed packages and optional published-package catalog.	No hidden alternate storage authority; IO and trust boundaries must remain explicit.

Decisions to preserve

Using official Typst syntax and Tinymist avoids a second, inevitably incomplete language implementation. Keeping exported PDF bytes separate from preview appearance prevents dark mode or raster quality from changing output. Fallbacks preserve the requested backend and expose the effective one rather than silently rewriting preferences. These are sound product-level boundaries.

The tradeoff is a larger process family, multiple coordinate systems and partially different views of unsaved dependencies. A service boundary is useful isolation, but it is not automatically a resource saving or a security sandbox.

What could improve

Represent the available capabilities as an explicit snapshot: editing, LSP features, interactive preview, PDF generation, rasterization and link extraction are related but not identical capabilities. A missing pdf to html should not be described as total compiler failure. Keep platform support and degradation reasons centralized instead of rediscovering them in each control.

Sources: Cargo.toml; README.md; src/main.rs; src/toolchain.rs; src/tinymist.rs; src/compiler.rs; src/app.rs:5825. The architecture notes under docs/architecture/ describe historical decisions.

2. Crates and dependency direction

There are three important Rust targets, plus a separate developer tool:

tiptotyp executable

- > tiptotyp library: projection/document adapter and theme APIs
- > tiptotyp-core: deterministic state and coordinate rules
- > GUI, native platform, filesystem and external-service adapters

tiptotyp library -> tiptotyp-core + syntax/theme dependencies

tiptotyp-core -> serde only

xtask -> packaging, sidecar verification and profiling orchestration

The root package contains both the executable and reusable library. `core` is a workspace member; `xtask` is excluded from that workspace and is tested separately. “In the library” does not mean “in the dependency-minimal core”: the distinction is intentional and important.

A genuinely useful core

`tiptotyp-core` owns document snapshots/history, save receipts, close approval, workflow phases, preview provenance, connection/recovery, pairing, scheduling, text coordinates and geometry. Callers supply time. No GUI, filesystem, process, network or thread work belongs there. Its generic cursor type lets the application use egui selections without making the core depend on egui.

Tests enforce the dependency allowlist and reject several classes of effect API. This is stronger than merely naming a folder “core.” State-machine tests can drive stale completions, retry deadlines and Unicode boundaries without a desktop.

Where modularity is still shallow

The binary declares a broad set of peer modules. Several `app/*` files use `super::*` and implement methods on the same `EditorApp`, gaining access to its complete state. Splitting a file this way helps navigation, but does not establish independent ownership. By contrast, `settings_panel` and `tooltips` are explicitly prevented from taking the whole application or starting its services.

`src/app.rs` has 14,130 lines in this checkout, including helpers and some test support; another 6,906 lines are in `src/app/tests.rs`. `tinymist.rs` has 4,423 lines, including its tests. These are review-surface indicators, not production LOC counts or proof of poor code. The stronger evidence is the breadth of fields and effect ordering in `EditorApp::ui_in_window`.

Recommended direction

Keep a small core; do not move Typst parsing, subprocesses or rendering into it to make the executable smaller. Extract cohesive modules with narrow input/output contracts first. Create another crate only when a dependency boundary, independent consumer or build/test need justifies it. Prefer concrete owners and typed actions over a generic service locator or application-wide event bus.

Sources: `Cargo.toml`; `core/Cargo.toml`; `core/README.md`; `src/lib.rs`; `core/src/lib.rs`; `tests/architecture_boundaries.rs`; `src/app.rs`:1077,8473.

3. Startup, process and windows

Startup is explicit, but launch policy is oddly placed

`main` initializes optional profiling, parses launch options, validates capture themes, constructs native options and channels, then creates an `AppShell` wrapped by `ScreenshotApp`. On macOS it creates `winit`'s event loop before augmenting the existing application delegate for Finder opens and menu commands. It does not replace `winit`'s lifecycle integration. The exact `winit` version pin documents that coupling. Normal file/workspace launch and deterministic capture modes share `LaunchOptions` inside `screenshot.rs`. Even invalid launch configuration is reported with screenshot-oriented wording. A small launch/configuration module would make the product entry point clearer without replacing the working capture infrastructure.

One UI thread, independently scheduled windows

`AppShell` owns primary and secondary document hosts, active-window selection, process-wide requests, shared preferences and process-close coordination. Each document window owns an `EditorApp`. Native UI still runs on the event-loop thread; deferred windows avoid repainting parents and siblings just because one window receives wheel input. Independent repainting is not parallel UI execution.

`DocumentHost` uses `Rc<RefCell<...>>` on that thread. A deferred callback captures a non-reused token, not an unsafe `Send` editor. The token resolves through a thread-local weak registry, asserts the correct thread, and becomes inert after host removal. Tests cover stale callbacks and unrelated-window repaint isolation. This is a well-targeted solution to `egui`'s callback interface.

Close is not quit

On macOS, closing the last document window retains a dormant native root so the application can reopen windows or Settings safely. `DocumentLifecycle` separates Active, Dormant and ResumePending. Queued document/preference changes settle before services restart. Process quit collects version-specific approval across windows and waits for protected operations. Empty tabs, a dormant window and process exit are three distinct states and must remain so.

Improvements and constraints

The shell should remain the owner of process-wide routing, not acquire document editing logic. Document hosts should expose a compact signal/event interface. `RefCell` borrowing and synchronous callbacks deserve re-entrancy tests, not an automatic conversion to mutexes. Mutexes do not make native handles thread-safe.

Make per-window disposable resources explicit, and release them separately from the retained native host. The companion resource audit found low dormant activity but substantial retained memory; it did not attribute that memory to this registry or prove a leak. Never "fix" it by destroying the current native view prematurely.

Sources: `src/main.rs:68`; `src/windowing.rs:68,214,353,429,690`; `src/windowing/document_host.rs:23,114`; `src/app/lifecycle.rs`; `src/app.rs:1710`; `src/native_window.rs`; `docs/multi-window-audit.md`.

4. Sessions, tabs and ownership

The app currently separates three choices that users experience independently: which native window is active, which tab is being edited, and which document drives the preview. That is correct. Opening an image/PDF tab can replace the code pane without replacing the pinned Typst preview. Closing the last tab leaves a workspace view instead of closing the native window.

The active-slot design

`EditorApp.document` contains the active document. `Tabs.parked` is a vector of optional parked documents; the active slot is `None`. A parallel ID vector provides stable tab control identity, while active and preview are positional indices. Parked tabs retain document/history, folding, `TextEdit` state, autosave deadline and workspace. They do not retain a complete app or another compiler/server.

Switching tabs swaps those fields into the active owner, restores widget state, rekeys the document/folding state, remaps close approvals, moves private backings, and clears caret, completion, hover, search and workflow state. Reordering updates both vectors and remaps the two indices. Tests cover many of these invariants.

Good: parked tabs are much cheaper than windows; preview identity survives reordering; text history is independent; obsolete responses are rejected. **Risk:** every new per-document feature requires deciding whether to park, clear, rekey or share another field. Forgetting one can leak state between tabs even when the UI looks correct. The parallel vectors and active hole are representational invariants maintained by procedures rather than enforced by one owner.

A better next representation

Use a document store keyed by `TabId/DocumentId`, with a separate ordered ID list and optional active/preview IDs. Each document record owns its persistent editing state. The active widget borrows that record; selecting a tab does not move the document into a different logical owner. A window controller owns services and chooses their inputs through document handles.

This is not a proposal for one language server per tab or a global mutable buffer shared by all windows. Same-path buffers in different windows may contain different unsaved edits; that policy must remain explicit. Preserve stable document versions for asynchronous work and reject responses for a different activation/context where the UI contract requires it.

Workspace identity is another separate concept

The active editor's workspace can differ from the preview document's root. Tab code already carries both. Name these contexts explicitly in requests: "active document," "preview entry," "project root" and "Git repository root" should not be interchangeable `PathBufs` selected opportunistically from window state.

Sources: `src/app/tabs.rs:6,19,143,285,420`; `src/app/tabs_tests.rs`; `src/app.rs:2009,2034`; `docs/tabs-and-git-hunks.md`. Rewriting this representation is proposed issue A01, not a claim that current tab-switch tests fail.

5. Document truth and source domains

Identity travels with text

`DocumentKey` identifies window owner, document epoch and revision. A `DocumentSnapshot` binds an immutable source snapshot to that key, rather than allowing consumers to assemble unrelated text and version values. Edits pass through a transaction; history operations also advance revisions. Save requests carry the snapshot actually written, and receipts acknowledge that snapshot. This is one of the strongest architectural foundations in the application.

Byte offsets, Unicode scalar positions and LSP UTF-16 positions have different meanings. Core text types and conversion functions make those boundaries explicit. Some feature-level APIs still use untyped `usize/Range<usize>` and local conversion helpers; typed coordinates have not yet reached every boundary.

miTeX is a projection, not a second saved format

The application document type wraps the core session in `mitex_document::Document<CCursorRange>`. The editor and undo own displayed text. A checked `CanonicalSnapshot` owns generated `Typst`, its matching key and a bidirectional source map. Plain documents take the unprojected route. Translation results, including failures, are cached per revision; coordinate indexes are lazy.

Displayed dollar notation becomes ordinary `mi/mitex` calls in `saved/service` source. Inline versus block math follows the whitespace rule. Binding checks fail closed for ambiguous/shadowed renderer names. Untouched supported calls preserve their original spelling. Unfinished input remains editable, but invalid translation is not sent to services or saved as an accidental different language.

The wrapper deliberately provides no mutable dereference to the core document. Service edits are preflighted and mapped atomically. Edits that change only hidden spelling can be refused because displayed-text undo could not restore them. These are principled constraints, not incidental inconvenience.

The remaining architectural burden

Every boundary must choose the right domain: editor features use displayed text; save, compilation, project analysis and Git comparison need canonical text; navigation and service responses need matching coordinate translation. Spreading this decision among many app methods creates omission risk.

Use request types that require a `CanonicalSnapshot` for service work and explicit editor coordinates for UI work. Preserve the no-projection-cost ordinary path. Do not unify the two snapshot types merely because their keys match. A registry rewrite must preserve save receipts and mode-change identity, not bypass them.

Tinymist preview navigation does not supply a document version for every click. Generation checks and valid mapping reduce risk but cannot prove the age of a click in an older rendered preview. Document that protocol limitation honestly.

Sources: `core/src/document.rs`; `core/src/text.rs`; `src/document.rs:4`; `src/mitex_document.rs:58,141`; `src/mitex_document/coordinates.rs`; `src/mitex_document/service_edits.rs`; `src/mitex_projection.rs`; `docs/mitex-projection.md`.

6. Saving, closing and durable effects

The core distinguishes a requested write, a committed receipt, an applied/stale receipt, and a synchronized versus uncertain disk outcome. Workflow phases separate queued actions, execution, prompts and dialogs. Close/Open continuations are not released merely because some earlier save finished. A newer edit or uncertain durability can keep the document open after bytes have reached disk.

Strong filesystem mechanics

`AtomicFileWriter` stages beside the destination, writes bytes, preserves existing permissions where available, synchronizes the temporary file, persists it, and synchronizes the parent. Destination-local staging avoids crossing mounts during replacement. A failure before commit is not conflated with uncertainty after commit. Private workspaces validate their location and naming and use scoped temporary objects. These details protect user data and are worth preserving exactly.

`resource_lock` serializes app-owned operations on a canonical resource across windows. It is a process-local mutex map with weak entries, not an OS lock against other editors. That distinction is documented in the code and should stay visible.

The orchestration is the weaker part

Active saves prepare a snapshot and call the atomic writer synchronously from `EditorApp`. Parked-tab autosaves have another synchronous loop. Opening and some other filesystem actions also perform direct IO in the application adapter. Native file choosers are asynchronous, but an asynchronous chooser does not make the following write asynchronous. Large files or slow storage can stall every window's UI thread.

Expected-disk fingerprint checks occur before entering the atomic writer's internal resource lock. The writer API accepts only path and bytes, not the expected state. It therefore cannot itself enforce the complete "check then commit" transaction. Today's synchronous UI limits some in-app overlap, but external writers can race; an asynchronous-save rewrite must not widen this gap. A process-local lock cannot provide a universal compare-and-swap against unrelated applications.

Proposed boundary

A `SaveCoordinator` accepts an immutable request containing document identity, destination, canonical bytes, expected disk state, intent and continuation token. A worker rechecks app-owned preconditions under the resource lease and returns a typed conflict, pre-commit failure, committed receipt, or durability warning. Only the UI owner applies the receipt and decides whether a continuation is valid.

Use the same transaction path for active save and parked autosave. Keep PDF export as a distinct artifact write policy. Closing an owner must not abandon a committed mutation; reuse the existing protected-operation completion mechanism. Characterize current save-close behavior before changing execution timing.

Sources: `core/src/document.rs`; `core/src/workflow.rs`; `core/src/closing.rs`; `core/tests/save_close.rs`; `src/app.rs:4004`; `src/app/tabs.rs:900`; `src/private_workspace.rs:254`; `src/resource_lock.rs`; `src/worker/exclusive.rs`.

7. The editor's internal pipeline

Native/egui input and focused command routing

- > TextEdit adapter, pairing and selection handling
- > document edit transaction / revision
- > invalidation and service/debounce scheduling
- > syntax, metrics, diagnostics and feature queries
- > layout, folding, gutter, overlays and popup presentation

Sensible use of the existing editor widget

egui TextEdit retains text input, IME, selection and basic cursor behavior. The app adds Typst parsing/highlighting, generic-language Syntect highlighting, completion, pairing, search/replace, code structure, fold controls, diagnostics and Git hunks. Inline diagnostic messages are painted decorations rather than inserted source. Folding keeps source character counts and changes visible galley rows instead of rewriting the document. These choices protect source/selection correspondence.

EditorDerivedData binds metrics, character indexes, syntax, regions, diagnostics, hover results and TeX completion data to document/query keys. Local TeX completion is intentionally structural and does not expand macros or call another server. Embedded Markdown uses CommonMark offsets; decoded string literals retain a map back to physical source. Literal asset navigation checks real syntax and workspace containment instead of treating arbitrary matching text as a file reference.

Coupling that merits controlled improvement

The editing path is spread across `editor_view`, app shortcut handlers, pairing, completion, formatting, search, table edits, Git revert and projection adapters. Each must produce one correct edit/selection/history outcome and invalidate the right data. The render/update callback also consumes service results, dialogs and commands before laying out controls; that ordering is part of behavior.

There are multiple syntax owners: highlighter, derived data and pairing maintain their own parsed state, while some context helpers parse probes. These are not necessarily redundant: a speculative pairing probe must not corrupt the shared real-document parse. Share an immutable versioned analysis snapshot for ordinary queries first, and keep speculative parses separate. Do not introduce a mutex-held global parser on the UI hot path.

The highlighter still constructs source text before its cache hit check and clones the cached layout job. Folding modifies egui galley internals. Neither demands an immediate custom editor rewrite, but both warrant allocation tests and dependency upgrade tests. A rope/delta history design should follow large-document evidence, not be bundled into ordinary controller extraction.

A focused design target

Introduce explicit edit intents/results at service and feature boundaries, carrying the source key, typed ranges, selection effect and undo grouping. Keep native typing/IME ownership in TextEdit. Extract feature renderers as read-only inputs plus actions, using Settings as the existing example; do not change all input paths at once.

Sources: `src/app/editor_view.rs`; `src/auto_pairs.rs`; `core/src/pairing.rs`; `src/editor_data.rs:74`; `src/highlight.rs:98`; `src/folding.rs`; `src/search.rs`; `src/completion.rs`; `src/tex_completion.rs`; `src/embedded_structure.rs`; `src/editor_features.rs`; `src/app.rs:2202,8473`.

8. Services and asynchronous work

Tinymist: several layers in one adapter

`tinymist.rs` contains subprocess supervision, stdout/stderr readers, bounded message framing, JSON-RPC serialization, session initialization, outstanding requests, feature decoding and preview commands. Typed generations prevent an old session's events from becoming current. Individual message/header limits and shutdown deadlines defend important failure paths; fake-server and real-tool tests exercise protocol behavior.

The service is not a generic LSP-client framework, and need not become one. A useful split is transport/framing, typed protocol DTOs/codecs, session state, and public commands/events. Preserve the current public interface during the first extraction. Keep preview-start failures distinct from unrelated formatting/navigation failures.

Compiler: artifact authority plus renderer coupling

The compiler worker maintains private source/watch context, reads CLI output, publishes PDF artifacts and raster results, and handles cancellation and shutdown. Its artifact identity separates source revision from generation. Canonical artifact availability does not have to wait for a successful raster upload.

Rasterization and PDF link extraction are currently housed in the compiler module and reused by asset loading. This makes PDF rendering look like a compiler detail even when the input is a directly opened asset. Extract a PDF raster service with its own page requests/resource policy, keeping Typst compilation responsible for canonical artifact production.

There are two kinds of work, correctly distinguished

Replaceable reads use latest-only mailboxes or `LatestJob`; stale results must not win. **Protected mutations** use `ExclusiveJob`, reject overlapping work on that owner and retain a completion if its window closes. A process-level active-operation lease prevents quit from abandoning such work. Preserve this distinction.

However, `LatestJob` dropping its receiver does not cancel computation already running. It spawns a thread per job. Latest-only pending queues do not establish a process-wide concurrency or byte budget. Some result/protocol channels are unbounded in aggregate. Compiler/Tinymist active loops also use short timed receives.

Recommended execution model

A modest bounded executor for replaceable jobs can provide keyed coalescing, cooperative cancellation, priority and completion routing. Keep long-lived protocol sessions under their own supervisors. Protected mutations need a separate admission and completion policy. Do not put both into a queue that silently drops “old” work.

Make every result's owner/version explicit. Shared repaint targets should wake only the relevant viewport; process-level completion should wake the shell. Avoid adding an async runtime solely to reorganize code unless native integration or measured IO scale actually requires it.

Sources: `src/tinymist.rs:647,943,1069,1469,3030`; `src/compiler.rs:149,434,779`; `src/worker.rs`; `src/worker/latest_queue.rs`; `src/worker/exclusive.rs`; `src/process.rs`; `src/asset.rs`.

9. Preview and native composition

Three concerns, currently interwoven

Policy: requested backend, effective backend, pause state, recovery budget and which document drives preview. **Content:** canonical PDF/image, artifact identity, current versus stale pixels. **Presentation:** native webview, raster geometry, textures, scrolling, color transforms and overlays.

The core already models canonical/raster provenance and connection/recovery. `PreviewController` holds application-facing preview state and diagnostics. The app still performs much of the service start/stop, URI/backing synchronization, navigation mapping and native-view coordination around it. A coordinator should make those transitions explicit without moving Wry handles into the core.

Good decisions

Old content may remain visible during compilation or recovery, but is not relabeled as current. Four delayed retries precede fallback on the fifth failed attempt; duplicate/stale failure events cannot consume extra budget. Explicit backend choice and automatic fallback remain separate. Preview status is in fixed chrome rather than changing document geometry as messages arrive.

The interactive route embeds Tinyquist's own served frontend; it does not implement the private vector protocol. Raster preview is continuous and pointer-anchored when zooming. Asset panes have a separate content controller, not another full compiler/Tinyquist stack. Export writes canonical PDF bytes rather than screen pixels.

The significant design debt

The raster path renders and retains all pages at 144 DPI, then uploads all textures. Visible-page geometry is not a resource-admission boundary. This is the clearest resource-related rewrite identified in the companion report. Split page metadata, artifact bytes, decoded pages and uploaded textures into separately owned stores; use page/scale/theme keys and bounded residency. Do not change export semantics.

Native child surfaces are outside egui's ordinary framebuffer clipping. The app uses `ChildViewSpec`, coordinate conversion and retained parent handles to manage focus, bounds and transparency. Popup viewports are needed in part because a normal egui overlay cannot reliably cover a native webview. This is real composition complexity, not merely decorative UI code.

Some hover state is held in viewport-keyed egui context data; app popup state and native-view state have other owners. Consolidate lifecycle cleanup and interaction state before adding more popup variants. Preserve movement-toward-popup handoff, scroll dismissal, keyboard access and no unsolicited focus stealing.

Native property updates can be diffed against the last applied state, but geometry correctness wins over shaving an unmeasured call. Any rewrite requires geometry tests plus native bounds traces and, when available, whole-window observation. A viewport PNG alone cannot prove native composition.

Sources: `core/src/preview.rs`; `core/src/connection.rs`; `core/src/recovery.rs`; `src/preview.rs`; `src/app/native_views.rs`; `src/app/raster_view.rs`; `src/child_view.rs`; `src/native_window.rs`; `src/app/tooltips.rs`; `docs/ui-qa-screenshots.md`; `companion resource-audit.typ`.

10. Workspace, Git and catalogs

Three different project views

WorkspaceTree is a sorted filesystem snapshot for Explorer. ProjectIndex follows literal local Typst imports/includes to derive headings, symbols, references and packages. Git has its own repository/status/diff model. These should not become one all-purpose tree: filesystem membership, language dependency and version-control state answer different questions.

The indexer does not execute Typst or download imports. It bounds traversal to 256 visited files and reports expressions it cannot resolve statically. This is safe, deterministic and honest about limits. Its result does not currently make every possible incompleteness equally explicit: unreadable files can be skipped and hitting the file cap ends traversal. A result with warnings/truncation metadata would distinguish “none found” from “not fully analyzed.”

Explorer’s whole-tree polling and per-window scans duplicate work. Build a shared workspace snapshot service with change notifications and immutable results, while keeping each window’s filters, selection and expansion state local. Overlay unsaved documents by identity/version for language analysis, not by altering the shared on-disk tree. Defer complex incremental indexing until basic invalidation is reliable.

Git: good transaction semantics, mixed layers

Git commands execute as argument arrays, not user-assembled shell snippets. The panel supports repository operations; the editor compares canonical buffer contents, including unsaved edits, with HEAD. Hunk Stage/Unstage modify the index, not the working file. Revert returns a checked replacement applied through normal editor undo. Partial staging preserves unrelated index changes and rejects unsafe overlaps/conflicts. This distinction is excellent and must survive refactoring.

git.rs nevertheless combines command execution, parsing, state, worker ownership and UI. git/editor/actions.rs also reaches upward through broad imports. Extract a repository service and pure hunk transaction model with typed inputs/results; leave the panel as a view/action adapter. General panel mutations already take a repository-root resource lock; the hunk path calls change_index directly. Route both through one repository transaction boundary. Git’s own locks and external changes still require conflict/error handling.

Fonts and packages are supporting services

Font discovery returns metadata and releases font-file bytes; system records are cached. Font samples use private, bounded atlases instead of reinstalling global UI fonts. Package discovery merges local installations with the published catalog, using a timeout and response-byte limit. Network failure can remain a warning while local data is useful. Uninstall targets one validated release and rejects linked or broad paths. These are well-scoped domain boundaries.

Improve explicit freshness/refresh and shared immutable catalogs before inventing a plugin system. Distinguish package catalog browsing from installation performed by Typst’s package machinery. Cache cleanup and decode limits belong in resource ownership, not in the rendering callback.

Sources: src/workspace.rs; src/project_index.rs:9,85; src/app/workspace_view.rs; src/git.rs; src/git/editor.rs; src/git/editor/actions.rs; src/font_catalog.rs; src/font_preview.rs; src/package_catalog.rs:241,631,866.

11. Preferences, commands and presentation

Settings already demonstrates the desired separation

AppSettings stores preferences and workspace history. settings_panel receives settings/read-only context and emits SettingsActions; it cannot mutate documents or run services. SettingsWindow owns a snapshot for an independently repainted viewport and returns edits/actions to its owner. AppliedPresentation compares a resolved request with the applied one to determine theme, font, scale, backend and tool effects. This prevents unrelated settings edits from triggering every effect.

Shared settings are merged at the shell. Independent field edits can coexist; the active owner wins a same-field conflict. Explicit workspace-history removals propagate before merging so a stale sibling cannot resurrect a removed entry. Malformed saved settings are reported and preserved as rejected input before valid preferences replace them. This is recovery, not a compatibility migration.

The remaining maintenance burden is manually coordinated settings snapshots, diffs, histories and side effects. Prefer a small typed change protocol separating global preferences, workspace history and window-local view state. Preserve current conflict semantics. Do not generate a generic reflective settings framework unless the simple protocol proves inadequate.

Themes are semantic, not isolated color patches

Built-in/imported palettes feed chrome, syntax, diagnostics and preview appearance. Sublime/TextMate import and whole-theme transforms live in reusable library modules. Typst-specific overrides inherit unset fields. Global font installation and atlas synchronization are restricted to named adapters; deferred viewports avoid the immediate-child atlas-copy repair path.

Keep this common palette pipeline. Add post-transform contrast diagnostics if needed: the transform module explicitly does not repair semantic contrast. Avoid an automatic correction that unpredictably changes a user's selected colors.

Commands are already centralized; targeting is harder

native_menu defines one command metadata list for native and in-app menus, including labels, requirements, sections and shortcut actions. The shortcut system handles overrides, normalization, platform bindings and more-specific chords. This is not a case for inventing another command registry. The complex part is choosing the target: active document, focused Settings text field, popup, hidden host or process. Clipboard actions must reach the focused widget rather than the source editor. No-window availability and native menu state have separate shell policy. A compact, tested CommandContext could make capability/target resolution explicit while retaining existing metadata.

Native tab dragging is another example of two input systems interacting: the tab owns egui drag behavior while an AppKit titlebar can still move the window. The scoped native drag guard restores prior state on release. Preserve that boundary and its native regression rather than replacing vector tab controls or egui input.

Sources: src/settings.rs; src/app/settings_panel.rs; src/app/settings_window.rs; src/presentation.rs:95; src/windowing.rs:307; src/native_menu.rs:30; src/shortcuts.rs; src/viewport_fonts.rs; src/theme_transform.rs; src/app/tabs.rs:63; tests/native_window_drag.rs.

12. Cross-cutting guarantees and delivery

Safety and trust boundaries

Rust targets deny unsafe code, with four explicit executable exceptions for `app_icon`, `native_window`, `native_menu` and `open_requests`. Keep unsafe operations adjacent to documented contracts; do not expand the allowlist to accommodate a controller refactor. The native delegate augmentation remains a dependency-upgrade hazard requiring tests around the pinned winit integration.

The app also handles untrusted files, images, fonts, package metadata and URLs. Path containment, symlink checks, message/response limits and literal command arguments are valuable defenses, but they do not constitute an OS sandbox. User-selected tool paths and fallback `PATH` executables are executable authority. Preview navigation stays embedded for the served origin and dispatches other targets through the application link policy. Keep that policy separate from UI rendering; do not expose arbitrary native operations to web content.

This was not an adversarial security test. Decoder limits, archive extraction, external-command capabilities and filesystem races merit targeted threat modeling when their boundaries change. Do not label structural concerns as proven exploits.

Error and observability design

Core outcomes and TinyMist errors are typed; many filesystem/Git/UI paths still flatten failures into strings. That is sufficient for a notice but weaker for recovery, retry classification and tests. Introduce typed errors at coordinator boundaries, then format at the UI edge; avoid a giant error enum covering every widget. Keep status history so transient messages need not all occupy the screen at once.

Profiling is opt-in and bounded, with normal spans compiled away. Snapshot scenes are non-persistent and owned by `QaSession`; capture routing and expected filenames are tested. However, capture mode can activate a raster path that production need not use. Resource/interaction tests must name the path they measure.

Build, packaging and test architecture

`xtask` fetches pinned sidecars, checks hashes and archive members, retains license/ provenance data, verifies package layout and runs profiling workloads. Runtime resolution prefers a valid custom selection, then bundled tools, then development environment/`PATH` fallback. Poppler and Git remain separate external dependencies. Runtime version constants and the sidecar manifest need coordinated updates; a single generated/validated manifest view would reduce drift.

CI defines formatting, strict Clippy and normal/profiling tests on Linux, macOS and Windows, plus a macOS real-tool job. This describes workflow configuration, not a claim that remote CI was run or passed during this audit. Local required checks were run for this report. Headless tests do not prove native focus, dragging, clipping or composed webview geometry.

The test portfolio is unusually strong for a young app: headless contracts, temporary-filesystem/Git tests, semantic egui tests, protocol fixtures, opt-in real tools and deterministic screenshots. Extend the same pattern at new seams. Source text checks are useful guardrails, not compiler-enforced isolation; dependency boundaries and public contract tests should carry the substantive guarantees.

Sources: `src/main.rs:1`; `src/lib.rs:3`; `src/process.rs`; `src/app.rs:11427`; `src/performance.rs`; `src/screenshot.rs`; `src/app/qa.rs`; `xtask/src/main.rs`; `toolchain/manifest.tsv`; `.github/workflows/checks.yml`; `tests/architecture_boundaries.rs`; `docs/architecture/0004-native-unsafe-boundaries.md`.

13. Scoped rewrites: document ownership

These issues are intentionally bounded. “Medium” and “large” describe integration surface, not promised duration. Each can ship independently with preserved behavior.

A01 · Stable document store for tabs

Priority: high. **Size:** large, staged. **Boundary:** Tabs, active document state and the window’s document accessors.

Replace the active-hole/parallel-vector representation with records keyed by stable IDs and a separate order list. Move folding, saved widget state and autosave metadata into the record. Keep window-owned services and active/preview selection separate. First introduce accessors and invariant tests; then change storage behind them.

Done when: reorder preserves active/preview identity; switching cannot lose history/selection or accept stale replies; empty-workspace and dirty-close tests pass; tab switching does not start extra services.

Exclude: shared mutable buffers across windows, ropes and per-tab servers. **Risk:** identity semantics and close approval. Measure active-switch allocations and idle many-tab work.

A02 · One asynchronous save transaction path

Priority: high. **Size:** medium. **Boundary:** active save and parked autosave, core receipts/workflows, atomic writer and protected-operation completion.

Introduce typed save requests/results and a coordinator. Perform disk work off the UI thread; recheck expected state within the app-owned resource transaction. Apply receipts only to their originating document/version and release a close continuation only after the existing durability rules permit it. Keep a record of completion if the window disappears.

Done when: slow-writer tests leave UI dispatch non-blocking; same-path saves serialize; stale/failed/uncertain writes cannot falsely close a document; active and parked paths use the same policy. **Exclude:** automatic conflict merging and claims of atomicity against unrelated external writers. **Risk:** data loss; start with adversarial save-order tests before changing execution timing.

A03 · Versioned document-service synchronization

Priority: high after identity contracts are clear. **Size:** medium.

Extract canonical snapshot collection, open URI tracking, private backing ownership and active/preview root selection into a coordinator. Its input is document handles plus a chosen preview entry; outputs are typed open/change/close commands and backing updates. Keep actual IO in adapters.

Done when: edit/switch/close/mode-change sequences emit exactly the intended updates; no canonical/displayed mixups; late generations are rejected; ordinary documents do no translation work. Explicitly retain the limitation that CLI compilation reads imported subfiles from disk, or address it in a separate issue. **Exclude:** silently changing multi-file unsaved compilation semantics. **Risk:** duplicate work and mismatched versions. Use command-log tests, not sleeps.

Primary seams: `src/app/tabs.rs`; `src/app.rs:2009,2106,2202,4004`; `src/mitex_document.rs`; `src/private_workspace.rs`; `core/tests/save_close.rs`.

14. Scoped rewrites: services and rendering

A04 · Preview coordinator with explicit effects

Priority: high. **Size:** medium. **Boundary:** preview policy and app orchestration.

Compose existing connection/recovery/content state into an application-level coordinator receiving events and producing start/stop/render/repaint effects. Distinguish requested backend, effective backend, artifact availability and native view readiness. Keep Wry and texture handles in presentation adapters.

Done when: deterministic sequences cover fifth-failure fallback, late readiness, pause, restart, export during recovery and preview-entry change. Rendering consumes a read-only status snapshot. **Exclude:** rewriting Tinymist's frontend or core retry rules. **Risk:** accidental extra compilations; assert one effect per transition and no idle repaint loop. Can start before A01 with current document handles.

A05 · PDF page service and bounded resident pages

Priority: high for resource efficiency. **Size:** medium-to-large, staged.

Extract PDF rasterization/link extraction from `compiler.rs`. Separate document page metadata from decoded page and texture caches. Introduce page/scale keys, visible-range requests, adjacent-page prefetch, cancellation and process-wide byte budgets. Compile/export continues to own canonical PDF bytes.

Done when: 1/20/100-page tests show bounded resident resources at fixed viewport; scroll/zoom stay responsive; old preview remains valid during replacement; theme changes cannot pair stale pixels with a new key. **Exclude:** renderer replacement and export-quality changes. **Risk:** blank pages on eviction or zoom. Compare the same optimized active/idle/cold/warm workloads from the resource report.

A06 · Split LSP framing from session behavior

Priority: medium. **Size:** small-to-medium for the first extraction.

Move bounded framing and JSON-RPC feature codecs into private transport/protocol modules; keep `TinymistSidecar` commands/events stable. Then isolate the session phase/pending-request logic from process supervision. Use explicit generation and document-version fields at the exposed seam.

Done when: existing fake/real-server tests pass; malformed/oversized/truncated frames and stale replies have deterministic tests; no UI dependency enters the codec. **Exclude:** a general LSP framework, changing protocols or replacing the threading model in the same patch. **Risk:** shutdown/event ordering. The initial move should have no material runtime cost; verify no new copies/queues.

Primary seams: `src/preview.rs`; `src/app/native_views.rs`; `src/compiler.rs:779`; `src/app/raster_view.rs`; `src/tinymist.rs:943,1069,3030`; `core/src/recovery.rs`; `core/src/connection.rs`.

15. Scoped rewrites: work and editor seams

A07 · Bounded execution for replaceable jobs

Priority: high under multi-window load. **Size:** medium.

Add a process-owned executor for selected read-only jobs with bounded concurrency, keyed pending replacement and cooperative cancellation. Migrate one job family first, such as project indexing. Carry its originating repaint target and source key. Reserve a separate protected path for mutations.

Done when: superseding 100 requests cannot run 100 obsolete jobs; cancellation is observed at bounded checkpoints; closed owners receive no UI result; queued payload bytes are bounded. **Exclude:** moving native views off-thread or replacing every long-lived worker. **Risk:** priority inversion/starvation; test scheduling deterministically and measure foreground latency while background work is busy.

A08 · Shared workspace snapshots and explicit completeness

Priority: medium-high. **Size:** medium. **Boundary:** scanning/indexing, not Explorer widget state.

Create one canonical-root workspace service with immutable snapshots and coalesced change notification. Keep window-local expansion/search separate. Replace routine whole-file checks with events plus a conservative verification fallback. Return index warnings and truncation explicitly; retain the literal-only dependency policy.

Done when: two windows do not duplicate an unchanged scan; atomic save/delete/ rename events refresh correctly; capped or unreadable indexes report incompleteness; unrelated windows keep their view state. **Exclude:** evaluating Typst or indexing downloaded package sources. **Risk:** missed changes; keep a tested fallback. Depends on A07 only if sharing its executor, not on a whole-app rewrite.

A09 · Typed edit boundary and focused feature views

Priority: medium. **Size:** medium, one feature at a time.

Begin with completion/formatting or table edits: accept versioned, typed source ranges and return one edit/selection/undo intent. Move its popup renderer to read-only inputs plus actions. Reuse core coordinate validation and the projection preflight boundary. Keep TextEdit responsible for native typing and IME.

Done when: Unicode/CRLF, invalid ranges, stale replies, selection and one-step undo tests pass; the renderer cannot borrow EditorApp or launch services. **Exclude:** replacing TextEdit, rewriting all shortcuts, merging speculative parser probes into the real parse, or introducing a rope. **Risk:** event ordering and cursor drift; semantic UI tests must accompany pure transaction tests. Measure unchanged-frame allocations and ensure the seam adds no full-source copies.

Primary seams: `src/worker.rs`; `src/worker/exclusive.rs`; `src/workspace.rs`; `src/project_index.rs`; `src/app/editor_view.rs`; `src/completion.rs`; `src/mitex_document/service_edits.rs`; `src/app/settings_panel.rs` as the existing extraction pattern.

16. Scoped rewrites: adapters and sequence

A10 · Repository service independent of Git UI

Priority: medium. **Size:** medium. Extract command runner, status/diff codecs and checked hunk transactions behind a repository handle. Keep panel/gutter views as consumers of immutable state and typed actions. Extend the existing panel-operation repository lock to the hunk transaction path and revalidate baselines before commit.

Done when: partial staging, unrelated staged changes, conflicts, new files, quoted paths, CRLF and no-final-newline tests pass; revert is still one editor undo; rendering cannot execute Git. **Exclude:** a Git implementation rewrite, automatic merging and new remote workflows. **Risk:** mutation scope; retain disposable-repo integration tests and protected completions.

A11 · Explicit viewport resource lifecycle

Priority: medium-high. **Size:** medium. Give child-view/popup caches a named owner and close/dispose hook; remove closed-viewport font sample slots and other owned context data. Diff applied native bounds/visibility where safe. Keep quick hide, durable close and dormant native hosting distinct.

Done when: repeated open/preview/close cycles return live cache counts to a baseline or plateau; late callbacks are inert; scroll dismissal, keyboard focus, clipping and native reopen tests pass. **Exclude:** destroying all hidden views immediately. **Risk:** composition regressions; use fresh native bounds traces and proportionate whole-window verification, not just framebuffer screenshots.

A12 · Launch/capability boundary and living architecture

Priority: medium, easy to stage. **Size:** small first patch. Move ordinary launch parsing out of screenshot ownership; expose capture as an explicit launch mode. Add a tool-capability snapshot, validate runtime version metadata against the sidecar manifest, and update obsolete current-state comments/ADRs.

Done when: normal/QA/profile launch tests preserve non-persistence rules; missing raster/link tools are separately observable; generated metadata cannot drift; docs accurately describe per-window sessions and conditional CLI work. **Exclude:** changing tool precedence, adding migrations or bundling new tools. **Risk:** startup/packaging compatibility; test before launching. Do not probe tools or read manifests every frame.

Suggested sequence

Start A12's documentation/launch cleanup and A06's behavior-preserving codec extraction. Characterize A01/A02 identities and save ordering next; land A02 in small stages. A04 and A05 then address the largest preview coordination/resource seams. A07/A08 reduce duplicated work. A09/A10/A11 can follow independently when their contracts are explicit. Do not undertake all of them in one branch.

Primary seams: `src/git.rs`; `src/git/editor/actions.rs`; `src/child_view.rs`; `src/font_preview.rs`; `src/screenshot.rs`; `src/toolchain.rs`; `xtask/src/main.rs`; `src/lib.rs`:9; `docs/architecture/0002-interactive-editor.md`.

17. How to carry out the rewrites

Preserve observable behavior before moving ownership

For each issue, first list the invariants and add a test at the proposed public boundary. Introduce the narrow interface with the old implementation behind it. Migrate one caller at a time, then remove the old path entirely. This repository is greenfield: do not leave permanent compatibility aliases, old settings keys or parallel implementations unless a migration is explicitly required.

Good boundaries make illegal calls difficult: a renderer cannot save; a service request cannot accidentally contain displayed projection text; a write completion cannot name an unrelated document; a closed viewport cannot retain an accessible native handle. Moving methods into a new file while preserving unrestricted access to `EditorApp` does not achieve that.

Do not over-abstract

Keep a concrete application shell, document store, service coordinators and adapter modules. A universal event bus, dynamic dependency injection, “manager” for every field, or plugin abstraction would add indirection without solving the observed ownership problems. An explicit event/effect enum is appropriate for a stateful coordinator; it need not become a framework for every paint operation.

Keep distinct state machines composed rather than merged into one enormous enum: document lifetime, save workflow, process close, service recovery and preview content have different triggers and guarantees. Their interfaces should specify the small facts they exchange.

Acceptance gates for every architectural change

- **Correctness:** existing contract tests plus adversarial ordering at the new seam. Include cancellation, late completion, close/reopen, mode change and two windows.
- **Performance/resources:** no new idle work, no per-frame disk access, bounded allocation/queue ownership, and no feature cost on the disabled path. For hot-path changes, compare matching optimized active/idle/cold/warm runs.
- **Native UI:** geometry and focus tests; fresh captures for material visual changes; native bounds traces when composition changes. Headless success is not desktop QA.
- **Maintainability:** fewer owners of the changed state, a documented interface and removal of superseded fields/helpers. Line count reduction alone is not a goal.

Documentation debt is architectural debt

The current library comments still describe `miTeX` controls as not enabled, while the runtime supports them. The older interactive-editor ADR describes Settings replacing the workspace, one TinyMist per workspace and an always-authoritative watch pipeline; current code has a separate Settings viewport, per-window sessions and conditional CLI work. Preserve these records as history with explicit superseding notes, and publish one concise current ownership/capability map.

Bottom line: the app already contains the right patterns. Extend the core’s versioned contracts and Settings’ focused inputs/actions to the window controller’s remaining responsibilities. Prioritize document identity, durable effects and preview resources; leave framework replacement out of scope.

Verification basis: `tests/architecture_boundaries.rs`; `core/tests/contracts.rs`; `core/tests/save_close.rs`; `src/app/tabs_tests.rs`; `docs/performance.md`; `AGENTS.md`. Prior resource measurements are in the companion `resource-audit.typ`; this report introduces no new runtime performance measurements.

18. Module map and audit provenance

This map covers the first-party application areas reviewed. Tests and generated assets are separate concerns; not every module needs its own crate or rewrite.

Area	Primary files / responsibility
Entry / shell	main, windowing, windowing/document_host, open_requests: launch, process requests, native windows and close routing.
Core rules	core/src/{document,workflow,closing,preview,connection,recovery,scheduling,text,geometry,pairing}: deterministic state, identities and validation.
Document domains	document, mitex_document and its coordinates / service_edits, mitex_projection: file-kind admission, source projection, canonical snapshots and transactional edits.
Window controller	app, app/{lifecycle,tabs,mitex_mode,extra_shortcuts}: per-window orchestration and tab/mode/input transitions.
Editing / analysis	editor_data, editor_features, auto_pairs, delimiters, folding, search, completion, tex_completion, embedded_structure, diagnostics, highlight, generic_highlight, rainbow, app/editor_view.
Services / IO	compiler, tinymist, asset, worker and its exclusive / latest_queue, process, resource_lock, private_workspace, toolchain.
Preview / native UI	preview, app/{raster_view,native_views,tooltips}, child_view, native_window, viewport_fonts: content, geometry, overlays, native handles and atlas ordering.
Workspace / Git	workspace, project_index, explorer, app/workspace_view, git, git/editor, git/editor/actions, app/git_actions.
Presentation / catalogs	settings, presentation, theme, syntax_theme, builtin_themes, sublime_theme, theme_transform, font_catalog, font_preview, unicode_fonts, package_catalog; Settings view/window/panel.
Commands / identity	native_menu, shortcuts, app_icon, window_logo: command metadata, platform bindings and app/window branding.
QA / delivery	screenshot, app/qa, performance/recording, app/tabs/trace, xtask/src/{main,profile}, integration tests, capture gallery script, CI, manifest, icons/fonts/licenses.

Checkout: 27f644cf8752fa75731e9003350f26f245016d6e, reviewed 17 September 2026. Existing resource-report artifacts and the user's todo.typ edits were preserved. File line counts/references describe this checkout and can move after a rewrite.

Method: manifest/module inventory, source and architecture-note review, tracing startup, edit, tab switch, save, service and close paths, and inspection of test/CI contracts. Third-party source internals were not comprehensively audited. Proposed designs and risks are reasoned from these interfaces, not claims of reproduced bugs.

Local checks: formatting, strict all-target Clippy, normal application/core tests and separate xtask tests. Exact commands/results are recorded alongside this report in architecture-audit-checks.md. The report PDF was compiled and its pages visually inspected. No remote CI, native interaction suite or new profiling run is implied.